

```

Table "pg_temp_3.temp_users_transaction"
Column | Type      | Collation | Nullable | Default
-----+-----+-----+-----+-----
pk      | integer   |           | not null | generated always as
identity
username | text      |           | not null |
gecos    | text      |           |          |
email    | text      |           | not null |
Indexes:
    "temp_users_transaction_pkey" PRIMARY KEY, btree (pk)
    "temp_users_transaction_username_key" UNIQUE CONSTRAINT, btree
    (username)

```

3. You can see the structure of the `temp_users_transaction` table, so now commit the transaction:

```

forumdb=> commit work;
COMMIT

```

If you re-execute the `DESCRIBE` command `\d temp_users_transaction`, PostgreSQL responds in this way:

```

forumdb=> \d temp_users_transaction
Did not find any relation named "temp_users_transaction".

```

This happens because the `on commit drop` option drops the table once the transaction is completed.

Managing unlogged tables

We will now address the topic of unlogged tables. For now, we will simply note that unlogged tables are much faster than classic tables (also known as logged tables) but are not crash-safe. This means that the consistency of the data is not guaranteed in the event of a crash.

The following snippet shows how to create an unlogged table:

```

forumdb=> create unlogged table if not exists unlogged_users (
    pk int GENERATED ALWAYS AS IDENTITY
    ,username text NOT NULL
    ,gecos text
    ,email text NOT NULL

```

```
,PRIMARY KEY( pk )
,UNIQUE ( username )
);
CREATE TABLE
```



Unlogged tables are a fast alternative to permanent and temporary tables. This performance increase comes at the expense of losing data in the event of a server crash. If the server crashes after the reboot, the table will be empty. This is something you may be able to afford under certain circumstances.

Creating a table

We will now explore what happens behind the scenes when a new table is created. Also, for tables, PostgreSQL assigns an object identifier called an OID. We have already seen `oid2name` in *Chapter 2, Getting to know your cluster*. Now we will see something similar. An OID is simply a number that internally identifies an object inside a PostgreSQL cluster. Let's now see the relationship between the tables created at the SQL level and what happens behind the scenes in the filesystem:

1. To do this, we will use the OIDs and a system table called `pg_class`, which collects information about all the tables that are present in the database. So, let's run this query:

```
forumdb=> select oid,relname from pg_class where relname='users';
   oid | relname
-----+-----
 16389 | users
(1 row)
```

Here, the `oid` field is the object identifier field, and `relname` represents the relation name of the object. As seen here, the `forumdb` database is stored in the 16389 directory.

2. Now, let's see where the `users` table is stored. To do this, go to the 16386 directory using the following code:

```
postgres@learn_postgresql:~$ cd /var/lib/postgresql/16/main/
base/16386
```

Or if you are using the Docker image, execute:

```
postgres@learn_postgresql:~$ cd /postgres/16/data/base/16386
```

3. Once here, execute the following command:

```
postgres@learn_postgresql:~/data/base/16386$ ls -l | grep 16389
-rw----- 1 postgres postgres      0 Jan  3 09:13 16389
```

As you can see, in the directory 16386, there is a file called 16389. In PostgreSQL, each table is stored in one or more files. If the table size is less than 1 GB, then the table will be stored in a single file. If the table has a size greater than 1 GB, then the table will be stored in two files and the second file will be called 16389.1. If the users table has a size greater than 2 GB, then the table will be stored in three files, called 16389, 16389.1, and 16389.2; the same thing happens for the users_username_key index.



In PostgreSQL, each table or index is stored in one or more files. When a table or index exceeds 1 GB, it is divided into gigabyte-sized segments.

In this section, we've learned how to manage tables, and we've seen what happens internally. In the next section, we will learn how to manipulate data inside tables.

Understanding basic table manipulation statements

Now that you have learned how to create tables, you need to understand how to insert, view, modify, and delete data in the tables. This will help you update any incorrect entries, or update existing entries, as needed. There are a variety of commands that can be used for this, which we will look at now.

Inserting and selecting data

In this section, we will learn how to insert data into tables. To insert data into tables, you need to use the INSERT command. The INSERT command inserts new rows into a table.

It is possible to insert one or more rows specified by value expressions, or zero or more rows resulting from a query. We will now go through some use cases as follows:

1. To insert a new user in the users table, execute the following command:

```
forumdb=> insert into users (username,gecos,email) values
('myusername','mygecos','myemail');
INSERT 0 1
```

This result shows that PostgreSQL has inserted one record into the users table. The first number is the OID of the row that has been inserted; newer versions of PostgreSQL by default have tables created without OIDs on the rows, so you just get a 0 returned.

2. Now, if we want to see the record that we have just entered into the users table, we have to perform the select command:

```
forumdb=> select * from users;
 pk | username | gecos | email
----+-----+-----+-----
  1 | myusername | mygecos | myemail
(1 row)
```

The select command is executed in order to retrieve rows from a table. With this SQL statement, PostgreSQL returns all the data present in all the fields of the table. The value * specifies all the fields present. This can also be expressed as follows:

```
forumdb=> select pk,username,gecos,email from users;
 pk | username | gecos | email
----+-----+-----+-----
  1 | myusername | mygecos | myemail
(1 row)
```

3. Let's now insert another user into the users table; for example, insert the user 'scotty' with all their own fields:

```
forumdb=> insert into users (username,gecos,email) values
('scotty','scotty_gecos','scotty_email');
INSERT 0 1
```

4. If we want to perform the same search as before, ordering data by the username field, we have to execute the following:

```
forumdb=> select pk,username,gecos,email from users order by
username;
 pk | username | gecos | email
----+-----+-----+-----
  1 | myusername | mygecos | myemail
  2 | scotty | scotty_gecos | scotty_email
(2 rows)
```



The SQL language, without the ORDER BY option, does not return the data in an orderly manner.

In PostgreSQL, this could also be written as follows:

```
forumdb=> select pk,username,gecos,email from users order by 2;
pk | username | gecos | email
----+-----+-----+-----
 1 | myusername | mygecos | myemail
 2 | scotty | scotty_gecos | scotty_email
(2 rows)
```



PostgreSQL also accepts field positions on a query as sorting options.

5. Let's now see how to insert multiple records using a single-row statement. For example, the following statement will insert three records in the categories table:

```
forumdb=> insert into categories (title,description) values ('C
Language', 'Languages'), ('Python Language','Languages');
INSERT 0 2
```

This is a slight variation of the INSERT command. Our categories table will now contain the following values:

```
forumdb=> select * from categories;
pk | title | description
----+-----+-----
 1 | Database | Database related discussions
 2 | Unix | Unix and Linux discussions
 3 | Programming Languages | All about programming languages
 4 | C Language | Languages
 5 | Python Language | Languages
(5 rows)
```

6. Now, if we want to select only the tuples where the description is equal to Database related discussions, use the where condition:

```
forumbd=> select * from categories where description = 'Database
related discussions';
pk | title | description
----+-----+-----
1 | Database | Database related discussions
(1 row)
```

7. The where condition filters on one or more fields of the table. For example, if we wanted to search for all topics with title as orange and description as fruits, we would have to write the following:

```
forumbd=> select * from categories where description = 'Languages'
and title='C Language';
pk | title | description
----+-----+-----
4 | C Language | Languages
(1 row)
```

8. Now if, for example, we want to select all the tuples that both have a description field equal to Languages and are sorted by title in reverse order, execute the following:

```
forumbd=> select * from categories where description = 'Languages'
order by title desc;
pk | title | description
----+-----+-----
5 | Python Language | Languages
4 | C Language | Languages
(2 rows)
```

Or we could also write this:

```
forumbd=> select * from categories where description = 'Languages'
order by 2 desc;
pk | title | description
----+-----+-----
5 | Python Language | Languages
4 | C Language | Languages
(2 rows)
```



The ASC and DESC options sort the query in ascending or descending order; if nothing is specified, ASC is the default.

NULL values

In this section, we will talk about NULL values. In the SQL language, the value NULL is defined as follows:



Null (or NULL) is a special marker used in Structured Query Language to indicate that a data value does not exist in the database. Introduced by the creator of the relational database model, E. F. Codd, SQL NULL serves to fulfill the requirement that all true **Relational Database Management Systems (RDBMSs)** support a representation of missing information.

Now let's check out how NULL is used in PostgreSQL:

1. Let's start by inserting a tuple in this way:

```
forumbd=> insert into categories (title) values ('A new
discussion');
INSERT 0 1
```

2. Let's see now which tuples are present in the categories table:

```
forumbd=> select * from categories;
 pk |          title          | description
----+-----+-----
  1 | Database                | Database related discussions
  2 | Unix                    | Unix and Linux discussions
  3 | Programming Languages   | All about programming languages
  4 | C Language              | Languages
  5 | Python Language         | Languages
  6 | A new discussion        |
(6 rows)
```

3. So now, if we want to select all the tuples in which the description is not present, we use the following:

```

forumbd=> select * from categories where description = '';
pk | title | description
----+-----+-----
(0 rows)

```

As you can see, PostgreSQL does not return any tuples. This is because the last insert has entered a NULL value in the description field.

4. In order to see the NULL values present in the table, let's execute the following command:

```

forumbd=> \pset null NULL
Null display is "NULL".

```

5. This tells psql to show NULL values that are present in the table as NULL, as shown here:

```

forumbd=> select * from categories;
pk | title | description
----+-----+-----
1 | Database | Database related discussions
2 | Unix | Unix and Linux discussions
3 | Programming Languages | All about programming languages
4 | C Language | Languages
5 | Python Language | Languages
6 | A new discussion | NULL
(6 rows)

```

As you can see, the description value associated with the title A new discussion is not an empty string; it is a NULL value.

6. Now, if we want to see all records that have NULL values in the description field, we have to use the IS NULL operator:

```

forumbd=> select title,description from categories where description
is null;
title | description
-----+-----
A new discussion | NULL
(1 row)

```

The preceding query looks for all tuples for which there is no value in the description field.

- Now, we will search for all tuples for which there is a value in the description field using the following query:

```
forumdb=> select title,description from categories where description
is not null;
```

title	description
Database	Database related discussions
Unix	Unix and Linux discussions
Programming Languages	All about programming languages
C Language	Languages
Python Language	Languages

(5 rows)



To perform searches on NULL fields, we have to use the operators IS NULL / IS NOT NULL. An empty string is different from a NULL value.

Sorting with NULL values

Now let's see what happens when ordering a table where there are NULL values present:

- Let's repeat the sorting query that we performed previously:

```
forumdb=> select * from categories order by description ;
```

pk	title	description
3	Programming Languages	All about programming languages
1	Database	Database related discussions
4	C Language	Languages
5	Python Language	Languages
2	Unix	Unix and Linux discussions
6	A new discussion	NULL

(6 rows)

As you can see, all description values are sorted and NULL values are positioned at the end of the result set. The same thing can be achieved by running the following:

```
forumbd=> select * from categories order by description NULLS last;
pk |          title          |          description
----+-----+-----
 3 | Programming Languages | All about programming languages
 1 | Database              | Database related discussions
 4 | C Language            | Languages
 5 | Python Language       | Languages
 2 | Unix                  | Unix and Linux discussions
 6 | A new discussion      | NULL
(6 rows)
```

2. If we want to place NULL values at the beginning, we have to perform the following:

```
forumbd=> select * from categories order by description NULLS first;
pk |          title          |          description
----+-----+-----
 6 | A new discussion      | NULL
 3 | Programming Languages | All about programming languages
 1 | Database              | Database related discussions
 4 | C Language            | Languages
 5 | Python Language       | Languages
 2 | Unix                  | Unix and Linux discussions
(6 rows)
```



If not specified, the following are the default actions for ORDER BY type queries:

ORDER BY NULLS LAST is the default for ASC (which is also the default) and NULLS FIRST is the default for DESC.

Creating a table starting from another table

We will now examine how to create a new table using data from another table. To do this, you need to create a temporary table with the data present in the categories table as follows:

```
forumbd=> create temp table temp_categories as select * from categories;
SELECT 6
```